

Anyway, let's discuss how Python bindings help us execute ns-3 (the source code of which is in C++) with Python code. In this aspect, the use of Tcl in ns-2 and Python in ns-3 have some similarity. But the main difference is that the execution of an ns-2 simulation is possible only with Tcl scripts, whereas the execution of an ns-3 simulation is possible even without using Python scripting. The Python script shown below called *pythonscript.py* executes C++ based ns-3 simulation. This is a slightly modified version of a Python script available in the standard set of examples provided with ns-3. You can download the file *pythonscript.py* and all the other files mentioned in this article, from http://opensourceforu.efytimes.com/article_source_code/October2015/ns3.zip.

```
import ns.applications
import ns.core
import ns.internet
import ns.network
import ns.point_to_point
ns.core.LogComponentEnable("UdpEchoClientApplication",
ns.core.LOG_LEVEL_ALL)
ns.core.LogComponentEnable("UdpEchoServerApplication",
ns.core.LOG_LEVEL_ALL)
nodes = ns.network.NodeContainer( )
nodes.Create(2)
pointToPoint = ns.point_to_point.PointToPointHelper()
pointToPoint.SetDeviceAttribute("DataRate", ns.core.
StringValue("10Mbps"))
pointToPoint.SetChannelAttribute("Delay", ns.core.
StringValue("1ms"))
devices = pointToPoint.Install(nodes)
stack = ns.internet.InternetStackHelper( )
stack.Install(nodes)
address = ns.internet.Ipv4AddressHelper( )
address.SetBase(ns.network.Ipv4Address("192.168.100.0"),
ns.network.
Ipv4Mask("255.255.255.0"))
interfaces = address.Assign(devices)
echoServer = ns.applications.UdpEchoServerHelper(5000)
serverApps = echoServer.Install(nodes.Get(1))
serverApps.Start(ns.core.Seconds(2.0))
serverApps.Stop(ns.core.Seconds(8.0))
echoClient = ns.applications.UdpEchoClientHelper(interfaces.
GetAddress(1), 5000)
echoClient.SetAttribute("MaxPackets", ns.core.
UIntegerValue(1))
echoClient.SetAttribute("Interval", ns.core.TimeValue(ns.
core.Seconds(1.0)))
echoClient.SetAttribute("PacketSize", ns.core.
UIntegerValue(512))
clientApps = echoClient.Install(nodes.Get(0))
clientApps.Start(ns.core.Seconds(3.0))
clientApps.Stop(ns.core.Seconds(8.0))
ns.core.Simulator.Run( )
```

```
ns.core.Simulator.Destroy( )
```

The Python script *pythonscript.py* simulates a network similar to the one we have simulated with the ns-3 script *simulation1.cc*, which has two nodes connected by a point-to-point link, and a UDP echo server and client as applications. Please refer to Part 3 (OSFY, July 2015) of this series to refresh your knowledge about the simulation script *simulation1.cc*. There are two different types of Python bindings in ns-3. These are the monolithic bindings that provide API definitions for all the modules and can be found in the directory *ns/ns-allinone-3.22/ns-3.22/bindings/python*, and the modular bindings which provide API definitions for individual modules and can be found in the respective modules themselves.

The ns-3 APIs are included into a namespace by Python bindings. Earlier, Python bindings used to generate API definitions for ns-3 modules into a namespace called 'ns3'. Later on, Python bindings started generating API definitions for ns-3 modules into a namespace called 'ns'. You can observe that in the script *pythonscript.py*, the namespace called 'ns' is used. The rest of the Python script is similar to the ns-3 script *simulation1.cc*. The Python script creates two nodes connected by a point-to-point link. Then the data rate and delay associated with the link is set. IPv4 addresses are assigned to the two nodes with the following line of code:

```
address.SetBase(ns.network.Ipv4Address("192.168.100.0"),
ns.network.
Ipv4Mask("255.255.255.0"))
```

This convoluted statement to assign IPv4 addresses to nodes is necessary due to another drawback of Python bindings. Some of the conversion constructors in the C++ source of ns-3 are yet to be supported by Python bindings. Thus, we have to use explicit constructors like the one shown in the previous line of code. After assigning an IP address on both the nodes, a port number is assigned on the node containing the UDP echo server application. The port number assigned to the server application is 5000. After this, a UDP echo client is set up on the second node. UDP echo client parameters like the maximum number of packets sent, the packet size, packet sending intervals, etc, are defined next. The starting and stopping times of both the server and client applications are also set. The lines of code '*ns.core.Simulator.Run()*' and '*ns.core.Simulator.Destroy()*' start and stop an instance of the ns-3 simulator.

Copy the file *pythonscript.py* to your system for execution. I have copied the file into the directory *ns/ns-allinone-3.22/ns-3.22/scratch*. Now open a terminal in the directory *ns/ns-allinone-3.22/ns-3.22* and execute the following command: